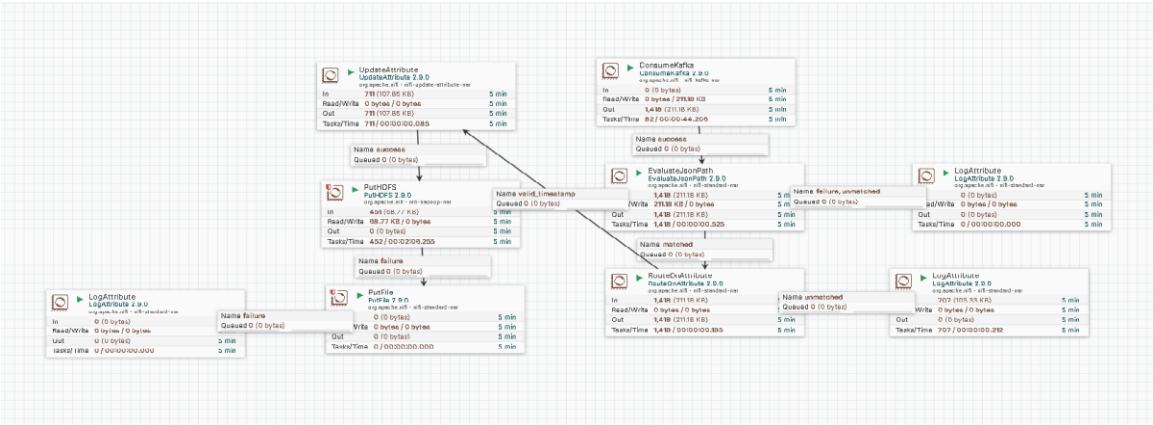
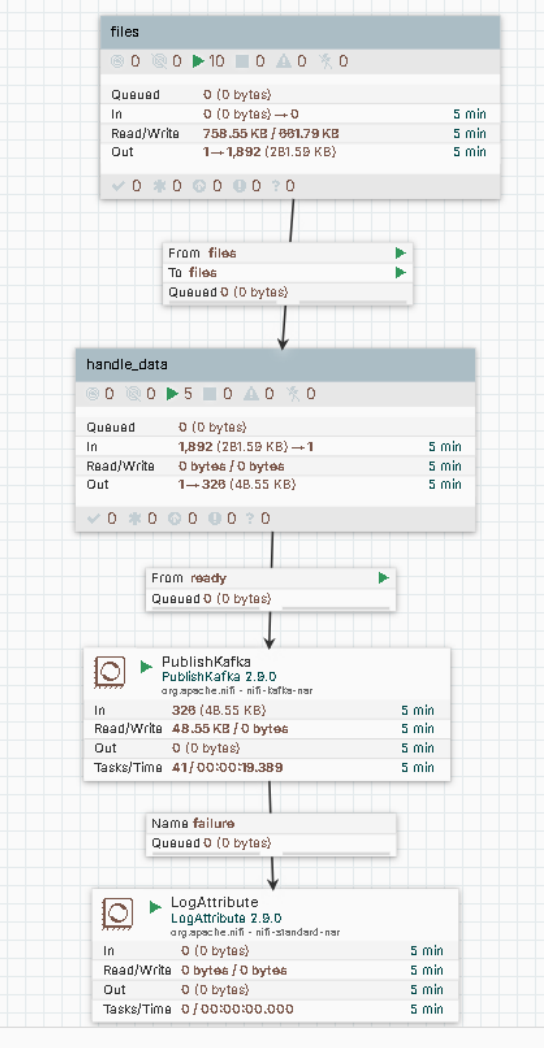
Data Engineering Assignment: Real-Time Data Pipeline Documentation

📋 Assignment Overview

This project implements a complete real-time data engineering pipeline using Apache NiFi, Apache Kafka, and Apache Hadoop (HDFS). The solution simulates streaming banking transaction data, processes it continuously, transforms CSV to JSON, and stores partitioned output in HDFS.

🏗️ Architecture Diagram





Part 1: Python Streaming Data Generator

Purpose

Generates continuous CSV data simulating real-time banking transactions with intentional data quality issues.

Messy Data Characteristics

Characteristic	How It's Simulated
Missing Values	Randomly omit customer_id, amount, or location
Duplicate Records	Same transaction_id appears multiple times (4% chance)
Invalid Records	Amount field contains text like "INVALID" or "abc"
Different Timestamp Formats	Mix of YYYY-MM-DD, YYYY/MM/DD, DD-MM-YYYY formats
Corrupted Rows	Rows with wrong delimiter format (2% chance)
Numeric Inconsistencies	Negative amounts, zero amounts, malformed decimals

Output Location

/opt/nifi/nifi-current/assignment2/stream_data/

Part 2: NiFi Producer Pipeline

Flow Summary

Processor	Function
GetFile	Reads CSV files from input directory every second
ConvertRecord	Transforms CSV to JSON using defined schema
SplitRecord	Splits files into individual records (1 record per FlowFile)
EvaluateJsonPath	Extracts fields as attributes for routing
RouteOnAttribute	Validates data, routes invalid records to error log
PublishKafka	Sends valid JSON messages to Kafka topic

Best Practices Applied

Concept	Implementation
Back Pressure	Queues configured at 10,000 objects / 1 GB
Yield Duration	1 second pause on failure
Penalization	30 seconds penalty for failed FlowFiles
Retry Handling	10 retries before routing to failure
Provenance Tracking	Data lineage tracking enabled
Queue Management	FlowFile expiration monitoring

Part 3: File Chunking

SplitRecord splits incoming batch files into individual records. Each record becomes a separate FlowFile (1 record per split). This enables:

- **Per-record validation**
- **Individual error handling**
- **Parallel processing**
- **Fine-grained Kafka message publishing**

Part 4: CSV to JSON Transformation

Input (CSV)

csv

transaction_id,customer_id,amount,transaction_type,location,timestamp

1001,CUST001,500.00,DEPOSIT,New York,2026-05-21 14:30:00

Output (JSON)

json

{

"transaction_id": 1001,

"customer_id": "CUST001",

"amount": 500.00,

"transaction_type": "DEPOSIT",

"location": "New York",

"timestamp": "2026-05-21 14:30:00"

}

Validation Rules

Validation	Condition	Failure Action
Missing amount	${amount:isEmpty()}	Route to Bad_Data_Log
Missing location	${location:isEmpty()}	Route to Bad_Data_Log
Missing customer_id	${customer_id:isEmpty()}	Route to Bad_Data_Log
Invalid amount format	Non-numeric amount	Route to Bad_Data_Log

Part 5: Kafka Integration

Topic Configuration

Setting	Value
Topic Name	bank-transactions
Partitions	3
Replication Factor	1

Producer (NiFi → Kafka)

- **PublishKafka sends JSON messages to the topic**
- **Delivery guarantee: all (acks=all)**

Consumer (Kafka → NiFi)

- **ConsumeKafka reads messages from the topic**
- **Group ID: hdfs-consumer-group**
- **Offset reset: earliest**

Connection

NiFi connects to Kafka brokers at: kafka1:19092, kafka2:19093, kafka3:19094

Part 6: HDFS Storage (Consumer)

Consumer Flow Summary

Processor	Function
ConsumeKafka	Reads JSON messages from Kafka topic
EvaluateJsonPath	Extracts fields as attributes
RouteOnAttribute	Validates timestamp format
UpdateAttribute	Extracts year, month, day from timestamp
PutHDFS	Writes files to HDFS with dynamic path

Partition Structure

/user/itversity/bank_transactions/year=${year}/month=${month}/day=${day}/

Browse Directory

/user/itversity/bank_transactions Go!

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	root	0 B	May 21 20:36	0	0 B	year=2026
drwxr-xr-x	root	root	0 B	May 21 20:36	0	0 B	year=2027

Showing 1 to 2 of 2 entries Previous 1 Next

Hadoop, 2020.

Browse Directory

/user/itversity/bank_transactions/year=2026 Go!

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	root	0 B	May 21 20:39	0	0 B	month=05
drwxr-xr-x	root	root	0 B	May 21 20:40	0	0 B	month=06
drwxr-xr-x	root	root	0 B	May 21 20:40	0	0 B	month=07
drwxr-xr-x	root	root	0 B	May 21 20:40	0	0 B	month=08
drwxr-xr-x	root	root	0 B	May 21 20:40	0	0 B	month=09
drwxr-xr-x	root	root	0 B	May 21 20:40	0	0 B	month=10
drwxr-xr-x	root	root	0 B	May 21 20:40	0	0 B	month=11
drwxr-xr-x	root	root	0 B	May 21 20:39	0	0 B	month=12

Showing 1 to 8 of 8 entries Previous 1 Next

Hadoop, 2020.

HDFS Output Example

/user/itversity/bank_transactions/

├── year=2026/

│ ├── month=05/

│ │ ├── day=11/

│ │ │ └── transactions_20260511_143022.json

│ │ └── day=12/

│ └── month=06/

Error Handling

- **HDFS write failures → Route to PutFile (local backup)**
- **Backup location: /opt/nifi/nifi-current/assignment2/failed_records/**

Part 7: Monitoring and Reliability

Error Handling Summary

Error Type	Location	Action
Missing amount	RouteOnAttribute	Log to Bad_Data_Log
Missing location	RouteOnAttribute	Log to Bad_Data_Log
Missing customer_id	RouteOnAttribute	Log to Bad_Data_Log
Invalid amount format	RouteOnAttribute	Log to Bad_Data_Log
Invalid timestamp format	RouteOnAttribute (consumer)	Log to LogAttribute

Error Type	Location	Action
Kafka connection failure	PublishKafka/ConsumeKafka	Retry with penalization
HDFS write failure	PutHDFS	Route to PutFile (local backup)

Logging

Multiple LogAttribute processors capture:

- **Failed record details**
- **Duplicate transactions**
- **Timestamp validation failures**
- **HDFS write failures**

Complete Data Flow Summary

Stage	Component	Action
1	Python Generator	Creates CSV files every 2 seconds
2	GetFile	Reads files from input directory
3	ConvertRecord	Transforms CSV to JSON
4	SplitRecord	Splits into individual records
5	EvaluateJsonPath	Extracts fields as attributes
6	RouteOnAttribute	Validates data quality

Stage	Component	Action
7	PublishKafka	Sends valid JSON to Kafka
8	Kafka	Stores messages in 3 partitions
9	ConsumeKafka	Reads JSON from Kafka
10	EvaluateJsonPath	Extracts fields from JSON
11	RouteOnAttribute	Validates timestamp format
12	UpdateAttribute	Extracts year/month/day
13	PutHDFS	Writes to partitioned HDFS